Lab Instructions
Merkle Trees in Blockchain

Objective:

By the end of this lab, students will:

1. Understand Merkle Trees and their role in blockchain systems for data integrity.
2. Implement a Merkle Tree in Python.
3. Integrate the Merkle Tree into an existing blockchain framework to enhance transaction validation and verification.

Lab Tasks:

Task 1: Introduction to Merkle Trees

Goal: Understand the role of Merkle Trees in blockchain and why they are used for efficient transaction verification.

1. **Understanding Merkle Trees**:
    - A Merkle tree is a binary tree where each leaf node is a hash of data (e.g., transactions), and each non-leaf node is the hash of its children.
    - The root hash of the Merkle tree summarizes all transactions in the block, providing a compact way to verify the integrity of all transactions.
2. **Use in Blockchain**:
    - Merkle Trees allow blockchain nodes to efficiently verify the integrity of transactions with only a small set of data (the Merkle proof), rather than downloading all transaction data.
    - This is crucial for scalability and performance in distributed systems.

Key concepts:

- Leaf nodes: Hash of individual transactions.
- Non-leaf nodes: Hash of concatenated child nodes.
- Root: Single hash that represents the entire set of transactions in a block.

Task 2: Implementing a Simple Merkle Tree

Goal: Implement the Merkle Tree from scratch and calculate the Merkle root for a set of transactions.

1. **Create a Function to Hash Transactions**:
    - Use SHA-256 to hash individual transactions.
2. **Build the Merkle Tree**:
    - Create leaf nodes by hashing transactions.
    - Pair adjacent leaf nodes, hash them, and move upwards in the tree until you have a single root hash.
    - If the number of leaves is odd, duplicate the last leaf to ensure all nodes have pairs.

Code Implementation:

```
import hashlib
import json

def sha256(data):
    """Returns the SHA-256 hash of the given data."""
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def build_merkle_tree(transactions):
    """Builds the Merkle tree and returns the Merkle root."""
    leaves =  . . .
    . . .
   return leaves[0]

# Example usage
transactions = ["tx1", "tx2", "tx3", "tx4"]
root = build_merkle_tree(transactions)
print(f"Merkle Root: {root}")
```

3. **Task Explanation**:

- **sha256**: A helper function to hash transaction data.
- **build_merkle_tree**: Constructs the Merkle tree and returns the root hash.

Task 3: Generating and Verifying Merkle Proofs

Goal: Learn how to generate and verify Merkle proofs to demonstrate transaction inclusion in the Merkle tree.

1. **Merkle Proof**:
    - A Merkle proof consists of the hashes along the path from a transaction to the root.

    - Given a transaction, you can use the proof to verify that the transaction is included in the Merkle tree without needing to download all transactions.
2. **Generate Merkle Proof**:
    - Traverse the tree from the given transaction to the root, collecting hashes of sibling nodes along the way.
3. **Verify Merkle Proof**:
    - To verify, concatenate the transaction with the provided proof and hash them iteratively until you reach the Merkle root.

Code Implementation:

```
def merkle_proof(transactions, target_tx):
    """Generate a Merkle proof for the given target transaction."""
    . . .
    . . .


    # Generate the proof by going from the bottom (leaves) up to the top
(root)
    proof = []
    . . .
    .  . .
    return proof

def verify_merkle_proof(proof, target_tx, root):
    """
    Verify a Merkle proof for the given target transaction.
    """
    .. .
    . . .
    return # True or False




# Example usage
transactions = ["tx1", "tx2", "tx3", "tx4"]
root = build_merkle_tree(transactions)
proof = merkle_proof(transactions, "tx3")

print(f"Merkle Proof for 'tx3': {proof}")
print(f"Verification: {verify_merkle_proof(proof, 'tx3', root)}")
```

4. **Explanation**:

- **`merkle_proof`:** Generates the proof for a given transaction.
- **`verify_merkle_proof`**: Verifies the proof by reconstructing the Merkle root.

Task 4: Integrating Merkle Tree into Blockchain

Goal: Modify the previous blockchain implementation to use the Merkle root in block headers for transaction verification.

1. **Add Merkle Root to Block**:
    - Instead of including a list of transactions directly, store the Merkle root in the block header.
2. **Modify Blockchain Class**:
    - Update the block creation to compute the Merkle root using the transactions in the block.

Code Update:

```
import hashlib
import datetime

def sha256(data):
    """Returns the SHA-256 hash of the given data."""
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

class Block:
    def __init__(self, index, timestamp, transactions, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.merkle_root = build_merkle_tree(transactions) # Store Merkle root
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        """Generate the block's hash."""
        block_string = str(self.index) + str(self.timestamp) +
self.merkle_root + self.previous_hash
        return sha256(block_string)

class Blockchain:
    def __init__(self):
```

```
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, str(datetime.datetime.now()), ["Genesis Block"],
"0")

    def add_block(self, transactions):
        last_block = self.chain[-1]
        new_block = Block(len(self.chain), str(datetime.datetime.now()),
transactions, last_block.hash)
        self.chain.append(new_block)

# Example usage
blockchain = Blockchain()
blockchain.add_block(["tx1", "tx2", "tx3"])
blockchain.add_block(["tx4", "tx5"])

for block in blockchain.chain:
    print(f"Block #{block.index}: {block.hash}")
    print(f"Merkle Root: {block.merkle_root}")
    print(f"Previous Hash: {block.previous_hash}")
    print()
```

3. **Explanation**:

- The **`Block`** class now includes the Merkle root.
- The blockchain's integrity is enhanced by using Merkle roots, ensuring efficient verification of all transactions in the block.

Task 5: Verification

Goal: Verify the integrity of the blockchain by comparing the Merkle root against the transactions in each block.

1. **Verify the Merkle Root**:
    - After adding a block to the chain, ensure the Merkle root is correct by verifying it against the list of transactions.

Code:

```
def verify_block(block):
    """Verifies that the block's Merkle root matches the transactions."""
   . . .

# Example usage
for block in blockchain.chain:
    if verify_block(block):
        print(f"Block #{block.index} is valid.")
    else:
        print(f"Block #{block.index} is invalid.")
```

2. **Explanation**:

- **`verify_block`:** Checks if the Merkle root stored in the block matches the root computed from the transactions.

Read more about Merkle tree:
https://decentralizedthoughts.github.io/2020-12-22-what-is-a-merkle-tree/